

Classification evaluation: the confusion matrix

1. What You'll Learn in This Section

In this lesson, you'll learn to:

- Explain how a decision tree picks its splits using the Gini index
- Split a dataset into training and testing portions the right way
- Calculate accuracy and recognize when it hides a poorly performing model
- Build a confusion matrix by hand and in Python, and read TP, TN, FP, and FN from it

2. Detailed Explanation

Recap: how decision trees pick splits with the Gini index

A decision tree grows by repeatedly picking the attribute (and threshold) that best separates the classes at each node. It measures "how mixed up" a node is using the Gini index, a number defined as:

$$\text{Gini} = 1 - \sum (\text{probability of class } i \text{ given the data at this node})^2$$

The summation runs over every unique class present in the data. A Gini value close to 0.5 signals a maximally confusing, impure node — with two classes, exactly 0.5 would mean a perfect 50/50 split where no confident decision is possible. A pure node is the opposite case: its Gini index is zero, meaning every instance in it can be classified correctly.

A worked example makes this concrete. Picture 8 students, each described by hours of study, with a pass/fail label — 5 passed and 3 failed.

Before any split, all 8 points sit at the root: $\text{Gini} = 1 - [(5/8)^2 + (3/8)^2] = 0.469$.

Testing the condition "hours ≤ 4 " sends 4 points left and 4 right.

Left branch (hours ≤ 4): fail, fail, fail, pass $\rightarrow \text{Gini} = 1 - [(1/4)^2 + (3/4)^2] = 0.375$.

Right branch (hours > 4): all four passed $\rightarrow \text{Gini} = 1 - [(4/4)^2 + (0/4)^2] = 0$ — a pure node.

Weighted Gini after the split combines both branches by how many points landed in each:

$$\text{Weighted Gini} = (4/8) \times 0.375 + (4/8) \times 0 = 0.188$$

$$\text{Reduction in impurity} = \text{Gini}(\text{before}) - \text{Weighted Gini}(\text{after}) = 0.469 - 0.188 = 0.281.$$

"Hours ≤ 4 " wins as the best split at this node because it produces the largest impurity reduction among all candidates tested. Other thresholds on hours (such as "hours > 5," "hours > 6," or "hours > 3") and conditions on other attributes are all compared, and whichever maximizes the reduction gets picked. After splitting on hours of study, the tree repeats the same process on the next attribute — for example, "attended," referring to class attendance — using the same Gini formula on the smaller subset of data that reached that branch.

Decision trees are considered highly self-explanatory: following the tree tells you exactly why a data point got its label. Other algorithms — logistic regression, linear regression, SVM, and neural networks — are much harder to explain this way, even though a developer can still build an intuitive sense of how they behave. Fully understanding the internal mathematics of neural networks from scratch generally needs a strong mathematics background.

From building models to evaluating them: splitting data into training and testing

So far two classification algorithms have come up: logistic regression and the decision tree. Evaluation metrics don't care which algorithm produced the predictions — the same metrics apply whether the model is a decision tree, logistic regression, SVM, or a neural network.

Before evaluating any model, the dataset is divided into a training portion and a testing portion. For example, 10,000 data points with an 80/20 split give 8,000 points for training and 2,000 for testing:

Input variables live in x , the output/target variable in y , exactly as with any toy dataset used earlier.

The model is fit using only the training portion of x and y .

Performance must never be reported on the training data, because the model has already seen those x , y pairs.

Performance is reported on the test portion instead: the fitted model receives the test x values only, produces predictions $\hat{y}$, and those predictions are compared against the withheld actual test y values.

Accuracy is computed with scikit-learn's `accuracy_score` function, called with the actual and predicted labels.

The training/testing ratio is flexible — 70/30 and 90/10 are common too, alongside 80/20. The general rule is that the majority of the data should go to training, because that's what lets the model "grasp the knowledge" in the data. Think of it like being handed several full machine-learning and NLP books versus being handed a short tutorial or a snippet of code — the books let you absorb far more. A model needs to see most of the data to learn well, and the held-out test portion just checks how much it actually learned. For very small datasets, an even higher training share (like 90/10) may be used since there isn't much data to spare.

Accuracy: the first evaluation metric

Accuracy is defined as:

```
Accuracy = (Number of correct predictions) / (Total number of instances)
```

A prediction is correct when the predicted label matches the actual label — an actual "pass" predicted as "pass," or an actual "fail" predicted as "fail." Using the 8-instance decision-tree dataset from the recap, the fitted tree classified 7 out of 8 instances correctly (only one "pass" instance was wrongly predicted as "fail"):

$$\text{Accuracy} = 7 / 8 = 0.875 \rightarrow 87.5\%$$

Accuracy is just one of several classification evaluation metrics. Precision and recall are always discussed as a pair, and F1 score is derived from them — F1 is the metric of choice when reporting results to a customer or in a publication. Sitting between these is the confusion matrix, an intermediate metric from which both accuracy and F1 score can be derived. It's especially valuable from a developer's point of view because it shows how well the model does for each individual class. That per-class view makes it possible to decide whether the model needs specific improvement for one particular class.

When accuracy misleads: imbalanced datasets

Accuracy has a major blind spot on imbalanced datasets — ones where the classes aren't present in roughly equal proportion.

Imagine a dataset of 10 students, 9 of whom failed and only 1 passed. A deliberately "blind" system with no learning at all is hard-coded to always predict "fail." Comparing its predictions against the actual labels, 9 out of 10 match purely because the dataset itself is skewed toward "fail":

$$\text{Accuracy} = 9 / 10 = 90\%$$

This blind, non-learning system could be presented as a "90% accurate solution," even though it contains no statistics, mathematics, or learned decision logic — it only exploits the class imbalance. The same trick works in reverse: if a dataset is mostly "pass," always predicting "pass" also scores highly without learning anything.

A dataset is balanced when it has roughly equal instances of each class (say, 5 pass and 5 fail out of 10); it's imbalanced when one class dominates. On a balanced dataset, a blind "always predict one class" system would only reach around 50% accuracy — correctly signaling that the system isn't working. On an imbalanced dataset, the same trick produces a misleadingly high number.

Watch out for: trusting accuracy alone. It's an acceptable standalone metric only for a balanced dataset. For an imbalanced dataset, always report it alongside precision, recall, F1 score, or the confusion matrix. Reporting these together is never harmful, and it reveals the truth about the data and the model's real performance either way.

The confusion matrix: structure and key terms

Instead of collapsing performance into one number, a confusion matrix arranges outcomes in a table. For binary classification it has two rows and two columns:

Rows represent the actual class (actual fail, actual pass).

Columns represent the predicted class (predicted fail, predicted pass).

For more than two classes, the matrix expands to $N \times N$ for N classes.

Each cell holds a count from comparing actual labels (y) against predicted labels ($\hat{y}$). The two diagonal cells are correct classifications; the two off-diagonal cells are errors.

To name the cells precisely, one class is designated the positive class — the outcome one is "positive about" or hoping for — and the other becomes the negative class. In a pass/fail example, "pass" is the natural positive class since passing is desirable. Which class is chosen as positive is a matter of convention and doesn't change the overall result, since the final metrics (accuracy, F1 score) summarize both classes together. With that convention:

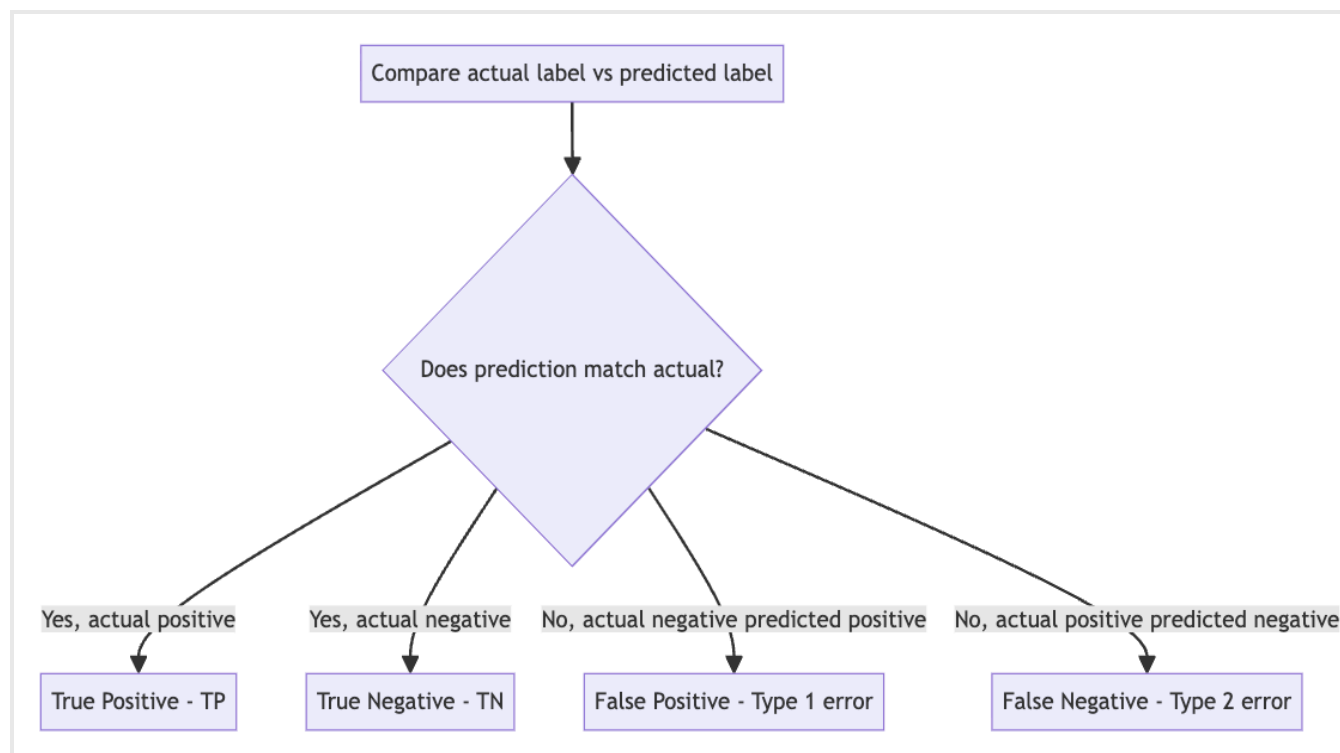
True Positive (TP): actual positive, predicted positive.

True Negative (TN): actual negative, predicted negative.

False Positive (FP): actual negative but predicted positive — a false alarm, also called a Type 1 error.

False Negative (FN): actual positive but predicted negative — a missed positive, also called a Type 2 error.

TP and TN sit on the diagonal as the desirable, correct outcomes; FP and FN are the two error cells.



Worked examples: filling in a confusion matrix by hand

Ten learners (pass/fail). With "pass" as positive and "fail" as negative, comparing actual versus predicted labels for 10 learners gives:

	Predicted fail	Predicted pass

Actual fail	3 (TN)	1 (FP)
Actual pass	1 (FN)	5 (TP)

Weather and the decision to play. A second, independent example uses weather conditions (sunny, cloudy, and so on) as input features, with "play" as the positive class (label 1) and "not play" as the negative class (label 0). Across 6 instances:

True Positives: 3 (actual play, predicted play)
 True Negatives: 1 (actual not-play, predicted not-play)
 False Positives: 1 (actual not-play, predicted play)
 False Negatives: 1 (actual play, predicted not-play)

This example reinforces a broader point: the confusion matrix only cares about the actual output (y) and predicted output ($\hat{y}$). Extra input features such as humidity or temperature — however many a model uses — never affect the confusion matrix itself.

Extending to multi-class classification

The confusion matrix generalizes beyond two classes. With three classes (class 1, class 2, class 3), the matrix becomes 3×3 — nine cells, with three "actual" rows and three "predicted" columns.

To compute true/false positive and negative counts per class, one class at a time is treated as "positive" while the rest count as "negative." For instance, if class 1 is positive, a point that is actually class 1 but predicted as class 2 counts as a false negative for class 1 — it was mislabeled as negative. The diagonal still represents the desired outcome (class 1 predicted as class 1, and so on), and every off-diagonal cell is an error relative to whichever class is currently treated as positive.

In practice, multi-class confusion matrices are built with scikit-learn rather than by hand, since manual bookkeeping gets unwieldy as the number of classes grows. Interpreting per-class true/false positives and negatives, and how that feeds into per-class accuracy or F1 score, is left for later. Understanding the binary case thoroughly comes first, since the multi-class case builds directly on it.

Setting up the Python tools and revisiting the blind spot in code

The implementation uses NumPy, pandas, and matplotlib, along with scikit-learn's metrics module:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
```

`confusion_matrix` computes the matrix itself; `ConfusionMatrixDisplay` renders it visually, and the two are always used as a pair — the raw matrix alone isn't directly viewable as a plot. Whenever a dataset is randomly generated or split, a fixed random seed of 42 keeps every run reproducible.

To reproduce the "blind system" scenario in code, an imbalanced dataset of 10 learners is created: nine pass (1) and one fail (0) in the actual labels. The predicted array is hard-coded to always say "pass," without running any real learning algorithm:

```
actual = np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 0])
predicted = np.array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])

accuracy_score(actual, predicted) # 0.90
```

The accuracy score comes out to 90% even though the "model" never learned anything and cannot predict the negative (fail) class at all — the same misleading pattern seen with the hand-worked example earlier.

Building, visualizing, and manually verifying a confusion matrix

A second, more balanced dataset of 10 learners is built as a pandas `DataFrame`:

```
demo = pd.DataFrame({
    "actual": [1, 1, 1, 1, 1, 0, 0, 0, 0, 1],
    "predicted": [1, 1, 1, 1, 0, 0, 0, 0, 1, 1],
})
```

Here 1 means "pass" and 0 means "fail." Reading the two columns by eye: the first four learners and the tenth are predicted correctly as pass; the fifth is actually pass but predicted fail (an error); learners six through eight are correctly predicted fail; the ninth is actually fail but predicted pass (another error).

Computing and plotting the confusion matrix with scikit-learn:

```
cm = confusion_matrix(demo["actual"], demo["predicted"], labels=[0, 1])

disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["fail", "pass"])
disp.plot(cmap="YlOrRd", colorbar=True)
plt.title("Confusion matrix: pass/fail prediction")
plt.show()
```

`labels=[0, 1]` tells scikit-learn explicitly which two labels to build the matrix around. The `cmap` parameter controls the plot's shading — larger counts appear darker, smaller counts lighter — and `colorbar=True` shows that scale alongside the plot (`False` removes the bar without touching the matrix values). Rows in the plot represent actual fail (top) and actual pass (bottom); columns represent predicted fail (left) and predicted pass (right). Running this on `demo` reproduces the manually counted result exactly: 3 true negatives, 1 false positive, 1 false negative, and 5 true positives.

Boolean filtering on the `DataFrame` pulls out exactly which learners were misclassified:

```
false_positive = demo[(demo["actual"] == 0) & (demo["predicted"] == 1)]
false_negative = demo[(demo["actual"] == 1) & (demo["predicted"] == 0)]

print(false_positive)
print(false_negative)
```

This prints the row where an actual fail was predicted as pass — the false positive, the ninth learner — and the row where an actual pass was predicted as fail — the false negative, the fifth learner.

To compute TP, TN, FP, and FN without calling `confusion_matrix` directly, a small helper function takes the actual array, the predicted array, and which label counts as positive:

```
def manual_confusion_matrix(actual, predicted, positive_label=1):
    actual = np.array(actual)
    predicted = np.array(predicted)

    tp = np.sum((actual == positive_label) & (predicted == positive_label))
    tn = np.sum((actual != positive_label) & (predicted != positive_label))
    fp = np.sum((actual != positive_label) & (predicted == positive_label))
    fn = np.sum((actual == positive_label) & (predicted != positive_label))

    return tp, tn, fp, fn
```

Applied to the imbalanced "blind system" arrays: `TP = 9, TN = 0, FP = 1, FN = 0`. That single false positive is the confusion matrix's way of flagging that the model completely fails on the negative class — something the 90% accuracy figure alone hid. The confusion matrix by itself still isn't the final verdict, though: F1 score more sharply exposes just how badly the model performs on that one class. Applied to the balanced `demo` dataset, the function reproduces the same counts scikit-learn gave: `TP = 5, TN = 3, FP = 1, FN = 1`.

Computing accuracy from confusion matrix counts

Once TP, TN, FP, and FN are known, accuracy can be computed directly from them, since correct predictions are exactly TP + TN and total instances are TP + TN + FP + FN:

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

For the balanced `demo` dataset: `Accuracy = (5 + 3) / 10 = 0.80` — matching the value `accuracy_score` produces directly on the same actual/predicted columns.

Comparing models: what comes after the confusion matrix

Two models are never compared directly by their accuracy alone, nor by staring at raw confusion matrix cell counts. Accuracy can't capture differences that arise specifically from an imbalanced dataset, and the confusion matrix on its own doesn't reduce cleanly to a single comparable score. Model comparison instead relies on a single derived number, the F1 score, which is computed from the confusion matrix and designed to expose weaknesses like a model being effectively "useless" on one class despite a high accuracy. Precision and recall feed into F1 score and are always discussed together. Deciding which error type — false positives or false negatives — matters more in a given situation is a judgment call tied directly to precision, recall, and F1 score.

3. Key Takeaways

A decision tree picks each split by choosing the condition that gives the largest reduction in Gini impurity, then repeats the process on the reduced data at each deeper level.

Always fit a model on training data only and measure performance on held-out test data — reporting accuracy on training data is misleading because the model has already seen it.

Accuracy is (correct predictions) / (total instances), but it can look great on an imbalanced dataset even when the model has learned nothing useful.

A confusion matrix breaks predictions into True Positive, True Negative, False Positive (Type 1 error), and False Negative (Type 2 error), showing performance per class instead of one blended number.

Accuracy can also be recovered from the confusion matrix as $(TP + TN) / (TP + TN + FP + FN)$, and the confusion matrix itself feeds into the next metric, F1 score, for comparing models.

Mental model: Think of the confusion matrix as a report card that grades a model separately on each class, instead of accuracy's single overall grade that can hide a complete failure on one class.